

Service Package

The `service` package contains the business logic of the application. It acts as a bridge between the controllers and the repository layer, handling validation, filtering, and complex updates.

Files

`TroubleTicketService.java`

The core service class for managing Trouble Tickets.

```
public class TroubleTicketService {

    private final TroubleTicketRepository repository;

    @jakarta.inject.Inject
    public TroubleTicketService(TroubleTicketRepository repository) {
        this.repository = repository;
    }

    public List<TroubleTicket> findAll(Map<String, String> filters) {
        List<TroubleTicket> list = repository.findAll();
        // Filtering logic...
        return list;
    }

    public TroubleTicket create(TroubleTicket ticket, String host) {
        // Validation and HREF generation...
        return repository.save(ticket);
    }

    public TroubleTicket update(String id, Map<String, Object> updates) {
        // Partial update logic using reflection-like Map processing...
        return repository.save(ticket);
    }
}
```

Line-by-line Explanation

- **Constructor Injection:** Uses `@Inject` to receive a `TroubleTicketRepository` instance.
- **`findAll(Map<String, String> filters)`:** Retrieves all tickets from the repository and applies stream-based filtering (e.g., by status or severity) based on provided query parameters.
- **`findById(String id)`:** Attempts to find a ticket and throws a custom `ResourceNotFoundException` if it fails, which is then caught by the global

exception mapper.

- **create(...)**: Validates mandatory fields (`description`, `severity`, `ticketType`). If validation fails, it throws a `BadRequestException`. It also generates the resource's `href` and initializes the status history.
 - **update(String id, Map<String, Object> updates)**: Implements the PATCH logic. It iterates through the update map, updates allowed fields, handles status change logic (including history tracking), and uses Jackson's `ObjectMapper` to convert complex nested objects (like `Note` or `Attachment`).
 - **delete(String id)**: Removes the ticket via the repository or throws an exception if it doesn't exist.
-

Analysis

Annotations

- **@jakarta.inject.Inject**: Used on the constructor to facilitate Dependency Injection. This makes the service easily testable by allowing mock repositories to be injected during unit tests.

Data Types

- **Map<String, Object> (in update)**: This is crucial for implementing the HTTP PATCH method correctly. It allows the service to distinguish between a field being “null” in the request (to be ignored) vs. “missing” from the request.
- **Stream API**: Extensively used in `findAll` for concise and readable collection filtering.
- **ObjectMapper**: Used for deep conversion of Map values into specific model classes, ensuring type safety even when receiving dynamic data.

Architectural Decisions

- **Service Layer Pattern**: By centralizing business logic here, we ensure that rules (like mandatory field validation or status change history) are applied consistently, regardless of whether the request comes from the REST API or another part of the system.
- **Custom Exceptions**: The service layer throws domain-specific exceptions (`BadRequestException`, `ResourceNotFoundException`) rather than JAX-RS `WebApplicationException`. This keeps the service layer decoupled from the web framework (JAX-RS).
- **Validation**: Mandatory fields are checked programmatically here to ensure data integrity before persistence.